

7주차

조인 (JOIN)

INNER · OUTER · SELF · CROSS JOIN

이번 주를 마치면

목표 ①	조인의 필요성과 카티전 곱의 개념을 이해한다
목표 ②	INNER JOIN으로 고객-주문-상품을 연결해 조회한다
목표 ③	LEFT/RIGHT의 차이를 알고 주문 없는 고객을 찾는다
목표 ④	SELF JOIN으로 직원-상급자 관계를 조회한다
목표 ⑤	조인 조건 누락으로 생긴 카티전 곱 오류를 진단한다

「누가 무엇을 얼마나 샀는가」 — 경영의 기본 질문은 전부 조인으로 답합니다

오늘 다루는 다섯 가지

1 조인의 개념

2 INNER JOIN

3 OUTER JOIN

4 SELF JOIN과 CROSS JOIN

5 조인 결과 행 수 이해하기

1. 조인의 개념

나뉜 테이블을 공통 컬럼으로 잇는다

```
1 SELECT order_id, cust_id, order_date FROM orders;
2 -- 1001 | C001 | 2026-01-10      ← C001이 누구인지 알 수 없다
3
4 SELECT o.order_id, c.name, o.order_date
5 FROM   orders o
6 JOIN   customers c ON o.cust_id = c.cust_id;
7 -- 1001 | 한가람 | 2026-01-10
```

1

1-2행

1

조인 없이

고객이 「번호」로만 보인다

2

3

4행

2

별칭으로 출처를

o.는 orders, c.는 customers

3

6행

외래키 = 기본키

주문의 고객번호 = 고객의 고객번호

정규화된 DB는 정보가 여러 테이블에 나뉘어 있다 — 조인은 주로 기본키-외래키 관계로 다시 잇는다

1.1 조인의 종류

여섯 가지 조인 한눈에 보기

조인	결과	용도
INNER JOIN	양쪽 모두에 매칭되는 행만	가장 일반적인 조인
LEFT OUTER JOIN	왼쪽 전체 + 매칭되는 오른쪽 (없으면 NULL)	주문하지 않은 고객까지 포함
RIGHT OUTER JOIN	오른쪽 전체 + 매칭되는 왼쪽	테이블 순서를 바꾼 LEFT와 동일
FULL OUTER JOIN	양쪽 전체 (MySQL 미지원 → UNION으로 구현)	양쪽 누락 데이터 모두 확인
CROSS JOIN	모든 조합 (카티전 곱)	조합 생성, 더미 데이터 만들기
SELF JOIN	같은 테이블을 자기 자신과 조인	계층 구조(직원-상급자) 탐색

다운유통의 고객·주문·상품·직원 정보는 전부 다른 테이블에 있다 — 조인 없이는 매출 보고서도 없다

2. INNER JOIN

양쪽 모두에 짝이 있는 행만 남는다

✓ 표준 문법 (권장) — INNER는 생략 가능

```
1 SELECT o.order_id, c.name, o.status
2 FROM orders o
3 INNER JOIN customers c ON o.cust_id = c.cust_id;
4 -- INNER 키워드는 생략 가능 → JOIN 만 써도 INNER JOIN
```

구식 문법 (WHERE 조인) — 권장하지 않음

```
1 SELECT o.order_id, c.name, o.status
2 FROM orders o, customers c
3 WHERE o.cust_id = c.cust_id;
```

실행으로 나열하면 일단 모든 조합이 만들어진다 — WHERE 줄을 빠뜨리면 150행이 나온다

2. INNER JOIN

조인 조건을 쓰는 네 가지 방법

쓰는 법	SQL	기억할 점
ON (기본형)	INNER JOIN customers c ON o.cust_id = c.cust_id	어떤 열끼리 잇는지
INNER 생략	JOIN customers c ON o.cust_id = c.cust_id	JOIN만 써도 INNER
USING	FROM orders o JOIN customers c USING (cust_id)	열 이름이 같을 때만
NATURAL JOIN	SELECT * FROM orders NATURAL JOIN customers;	비권장 — 조인 열이 숨는다

customers.name과 products.name처럼 같은 이름의 열은 「테이블별칭.컬럼명」으로 — 안 하면 Column is ambiguous

주문 명세 — 주문·고객·상세·상품 연결

```
1 SELECT o.order_id AS 주문번호,
2      c.name AS 고객,
3      p.name AS 상품,
4      i.qty AS 수량,
5      i.unit_price AS 단가
6 FROM orders o
7 JOIN customers c ON o.cust_id = c.cust_id
8 JOIN order_items i ON o.order_id = i.order_id
9 JOIN products p ON i.prod_id = p.prod_id
10 WHERE o.order_id = 1001;
11 /*
12 1001 | 한가람 | 무선 전기포트 | 1 | 42000
13 1001 | 한가람 | 유기농 원두 500g | 2 | 18500
14 */
```

1

2-3행
이름이 같은 두 열
c.name·p.name — 별칭 필수

2

7행
주문 → 고객

3

8행
주문 → 주문상세
주문 1건 : 상세 N건

4

9행
상세 → 상품

5

10행
행 조건은 WHERE
조인이 끝난 결과에서 1001번만

2.2 조인 + 집계

주문별 결제 금액

```
1 SELECT o.order_id,  
2        c.name                AS 고객,  
3        SUM(i.qty * i.unit_price) AS 결제금액  
4 FROM   orders o  
5 JOIN   customers c ON o.cust_id = c.cust_id  
6 JOIN   order_items i ON o.order_id = i.order_id  
7 GROUP BY o.order_id, c.name  
8 ORDER BY o.order_id;
```

3행

- ① 수량 × 단가를 합산
상세 행마다 계산해 주문별로

5행

- ② 고객 이름을 붙이기 위한 조인

6행

- ③ 금액에 필요한 상세 행

7행

- ④ 비집계 열은 모두 **GROUP BY**에
ONLY_FULL_GROUP_BY (6주차)

조인으로 표를 합친 뒤 **GROUP BY**로 주문별 합계를 구한다

3.1 LEFT OUTER JOIN

주문하지 않은 고객까지 조회

INNER JOIN — 주문 이력이 있는 고객만

```
1 SELECT c.name, o.order_id
2 FROM customers c JOIN orders o ON c.cust_id = o.cust_id;
3 -- 주문이 없는 고객(서지안)은 결과에서 사라진다
```



✓ LEFT JOIN — 모든 고객, 주문이 없으면 NULL

```
1 SELECT c.name, o.order_id
2 FROM customers c
3 LEFT JOIN orders o ON c.cust_id = o.cust_id;
4 /*
5   한가람 | 1001
6   한가람 | 1004
7   ...
8   서지안 | NULL
9  */
```

LEFT JOIN: 왼쪽(FROM) 테이블은 전부 남기고, 오른쪽에 짝이 없으면 NULL로 채운다

3.1 LEFT OUTER JOIN

안티 조인 — 「매칭되지 않는 행」 찾기

```
1 SELECT c.cust_id, c.name
2 FROM   customers c
3 LEFT   JOIN orders o ON c.cust_id = o.cust_id
4 WHERE  o.order_id IS NULL;
5 -- C010 | 서지안
6
7 SELECT p.prod_id, p.name
8 FROM   products p
9 LEFT   JOIN order_items i ON p.prod_id = i.prod_id
10 WHERE i.order_id IS NULL;
11 -- P010 | A5 스프링 노트
```

1 3행
모든 고객에 주문을 붙이고

2 4행
NULL이 된 고객만
이탈 방지 캠페인 대상

3 9-10행
주문된 적 없는 상품
재고 정리 후보

LEFT JOIN + WHERE 오른쪽컬럼 IS NULL — NOT EXISTS, NOT IN으로도 같은 결과(9주차)

3.2 OUTER JOIN에서 조건의 위치

같은 조건, ON에 두느냐 WHERE에 두느냐

✓ ① ON 절에 조건 — 고객 10명은 모두 남는다

```
1 SELECT c.name, o.order_id, o.status
2 FROM   customers c
3 LEFT   JOIN orders o
4       ON c.cust_id = o.cust_id AND o.status = '배송완료';
5 -- 배송완료가 아닌 주문은 NULL 처리되지만
6 -- 고객 10명은 모두 표시
```

✗ ② WHERE 절에 조건 — INNER JOIN처럼 동작

```
1 SELECT c.name, o.order_id, o.status
2 FROM   customers c
3 LEFT   JOIN orders o ON c.cust_id = o.cust_id
4 WHERE  o.status = '배송완료';
5 -- NULL 행이 조건에서 탈락
6 -- → 배송완료 주문이 있는 고객만 표시
```

NULL = '배송완료'는 참이 아니다 — 오른쪽 테이블의 조건은 ON에 두어야 바깥 조인의 의미가 유지된다

3.3 RIGHT JOIN과 FULL OUTER JOIN

RIGHT는 LEFT의 자리 바꾸기, FULL은 UNION으로

```
1 SELECT o.order_id, c.name
2 FROM orders o
3 RIGHT JOIN customers c ON o.cust_id = c.cust_id;
4 -- 동일한 결과: FROM customers c LEFT JOIN orders o ON ...
5
6 SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.cust_id = o.cust_id
7 UNION
8 SELECT c.name, o.order_id FROM customers c RIGHT JOIN orders o ON c.cust_id = o.cust_id;
```

3행

1 오른쪽이 기준
실무는 LEFT로 통일

6행

2 왼쪽 전체

7행

3 합치고 중복 제거

8행

4 오른쪽 전체

MySQL은 FULL OUTER JOIN을 지원하지 않는다 — LEFT 결과와 RIGHT 결과를 UNION으로 합친다

4.1 SELF JOIN

같은 테이블을 두 번 — 직원과 상급자

```
1 SELECT e.name AS 직원,  
2        e.position AS 직급,  
3        m.name AS 상급자  
4 FROM   employees e  
5 JOIN   employees m ON e.manager_id = m.emp_id;  
6 /*  
7 이지은 | 대리 | 김성호  
8 박준영 | 사원 | 김성호  
9 정다은 | 사원 | 최민서  
10 윤서준 | 사원 | 강태윤  
11 */
```

1

1행
e = 직원 역할

2

2행
m = 상급자 역할

3

4

5행
상급자 번호 = 상급자의 사번
INNER라 상급자 없는 직원은 빠짐

4
7-10행
결과 4행

employees.manager_id가 employees.emp_id를 참조하는 자기참조 구조 — 별칭을 다르게 붙여 구분한다

4.1 SELF JOIN

상급자가 없는 직원까지 — LEFT JOIN

INNER — 상급자 없는 직원은 빠진다 (4행)

```
1 SELECT e.name AS 직원,  
2        e.position AS 직급,  
3        m.name AS 상급자  
4 FROM   employees e  
5 JOIN   employees m ON e.manager_id = m.emp_id;
```



✓ LEFT — 기준 = 직원 전체 8명

```
1 SELECT e.name AS 직원, m.name AS 상급자  
2 FROM   employees e  
3 LEFT   JOIN employees m ON e.manager_id = m.emp_id;  
4 -- 8행 - 김성호·최민서·강태윤·조하늘의  
5 -- 상급자는 NULL
```

조하늘(E008)은 사원이데 manager_id가 NULL — 이런 데이터 이상을 발견하는 것도 조인의 실무 쓰임

4.2 CROSS JOIN

카티전 곱 — 의도했나, 실수했나

의도한 CROSS JOIN — 고객 × 상품 = 100행

```
1 SELECT c.name AS 고객, p.name AS 상품
2 FROM   customers c CROSS JOIN products p;
3 -- 추천 후보 매트릭스처럼 "모든 조합"이
4 -- 필요할 때만 의도적으로 사용
```

× 조인 조건 누락 — 15 × 10 = 150행

```
1 SELECT o.order_id, c.name
2 FROM   orders o, customers c;
3 -- 올바른 질의는 WHERE o.cust_id = c.cust_id
4 -- 조건이 필요
```

주문 1만 건 × 고객 1만 명 = 1억 행 — 결과 행 수가 터무니없이 많으면 조인 조건부터 의심한다

5. 조인 결과 행 수 이해하기

customers(10) ⋈ orders(15)

조인 종류	결과
INNER JOIN	15행 — 주문이 있는 고객만, 주문 수만큼 반복 (한가람은 3행)
LEFT JOIN (customers 기준)	16행 — 15행 + 주문 없는 서지안 1행(NULL)
CROSS JOIN	150행 — 10×15

카티전 곱의 결과 행 수 = 왼쪽 행 수 × 오른쪽 행 수

1:N 조인은 1쪽을 N번 반복한다

반복되는 1쪽

01

한가람은 주문이 3건이라
조인 결과에 3번 나온다

부풀려지는 COUNT

02

조인 후 COUNT(*)는
고객 수가 아니라 행 수

COUNT(DISTINCT)

03

「주문한 고객 수」는
COUNT(DISTINCT c.cust_id)

1:N 관계를 조인하면 1쪽 데이터가 N번 반복된다 — 조인 후 COUNT(*)는 값이 부풀려질 수 있다

조인에서 자주 만나는 네 가지

■ 증상

! Column is ambiguous

! LEFT JOIN인데 INNER처럼 나온다

! 결과 행이 터무니없이 많다

! 고객 수가 부풀려진다

■ 원인

→ 양쪽에 같은 이름의 열(name)

→ 오른쪽 테이블 조건을 WHERE에

→ 조인 조건 누락 → 카티전 곱

→ 1:N 조인으로 1쪽이 반복

■ 해결

✓ 테이블별칭.컬럼명으로 한정

✓ 그 조건을 ON 절로 옮긴다

✓ ON 또는 WHERE 조인 조건 확인

✓ COUNT(DISTINCT c.cust_id)

확인 퀴즈

LEFT JOIN을 사용했는데 결과가 INNER JOIN과 같아졌다. 가장 흔한 원인은?

① ON 절에 조인 조건을 두었기 때문

② WHERE 절에 오른쪽 테이블의 조건을 두었기 때문

③ ORDER BY를 사용했기 때문

④ 테이블 별칭을 사용했기 때문

정답 및 해설

정답

② WHERE 절에 오른쪽 테이블의 조건을 두었기 때문

해설

WHERE 절에 오른쪽 테이블 조건을 두면 NULL 행이 조건에서 탈락하여 INNER JOIN과 같은 결과가 됩니다. 조건을 ON 절로 옮겨야 합니다.

핵심 요약

핵심 ①	INNER는 양쪽 매칭 행, LEFT는 왼쪽 전체 + 매칭 오른쪽
핵심 ②	OUTER JOIN의 오른쪽 조건은 ON에 — WHERE면 INNER가 된다
핵심 ③	안티 조인: LEFT JOIN ... WHERE 오른쪽.컬럼 IS NULL
핵심 ④	SELF JOIN은 같은 테이블에 다른 별칭 — 직원-상급자
핵심 ⑤	조인 조건을 빠뜨리면 카티전 곱(행 수 = $M \times N$)

주문 없는 고객은 이탈 방지 캠페인, 미주문 상품은 재고 정리 후보

다음 주차 예고

8주차 · 중간고사

01

범위

1~7주차 전체

SELECT-집계-조인 중심

02

100점 통과제

학습사이트 중간평가

재응시 무제한

03

캡처 제출

100점 화면 캡처

15점

04

학습 정리

향상·학습 정리

(정량·정성) 5점

7주차 마무리

학습 마무리

각 주차의 실습 문제를 다시 풀어 보며 중간고사를 준비해 주세요

✓ INNER JOIN과 다중 테이블 조인

✓ LEFT JOIN과 안티 조인

✓ ON과 WHERE — 조건의 위치

✓ SELF·CROSS JOIN과 결과 행 수

이어서는 활동

7주차 실습 · 퀴즈 6 · 중간고사 준비